

GNN Applications: Graph for Protein Design

Jiaxuan You

Assistant Professor at UIUC CDS



CS512: Data Mining Principles, 2025 Fall

<https://ulab-uiuc.github.io/CS512/>

Logistics: Submission Task Due

- The deadline for the submission task is **Nov 21 (Friday), 11:59 PM CT**.
- We expect a minimum length of **7 pages** in **ICLR format** for the **draft submission**.
 - For the draft version, you are expected to include at least sections such as related work, methods, and **experiment settings**.

- We will use the **OpenReview** to receive submissions:

CS598 Deep Learning with Graphs 2024 Workshop

CS598 DLG 2024

🌐 Urbana, Illinois, United States 📅 Dec 04 2024 🔗 <https://ulab-uiuc.github.io/CS598/> ✉ cs598-you@siebelschool.illinois.edu

Please see the venue website for more information.

Submission Start: Sep 13 2024 11:59PM UTC-0, Submission Deadline: Dec 01 2024 11:59PM UTC-0

Add: **UIUC Fall 2024 CS598 DLG Submission**

- The draft project submission counts **10% of final grade**, remaining **15% is from the final version** (Due Dec 10)

Logistics: Submission Task Due

- The submission task counts towards **15% (writing) + 15% (implementation) = 30%** of your final grade.
 - For the 15% of writing, **only 5%** is determined by the draft version (due on **Nov 21**) and **10%** is determined by the final version (due on **Dec 8**).
 - The 15% of implementation is determined by the code you provide for the final version (due on **Dec 8**).

GNN Applications: Graph for Protein Design

Basics of Protein Design



Ill. Niklas Elmehed © Nobel Prize
Outreach

David Baker

Prize share: 1/2



Ill. Niklas Elmehed © Nobel Prize
Outreach

Demis Hassabis

Prize share: 1/4



Ill. Niklas Elmehed © Nobel Prize
Outreach

John Jumper

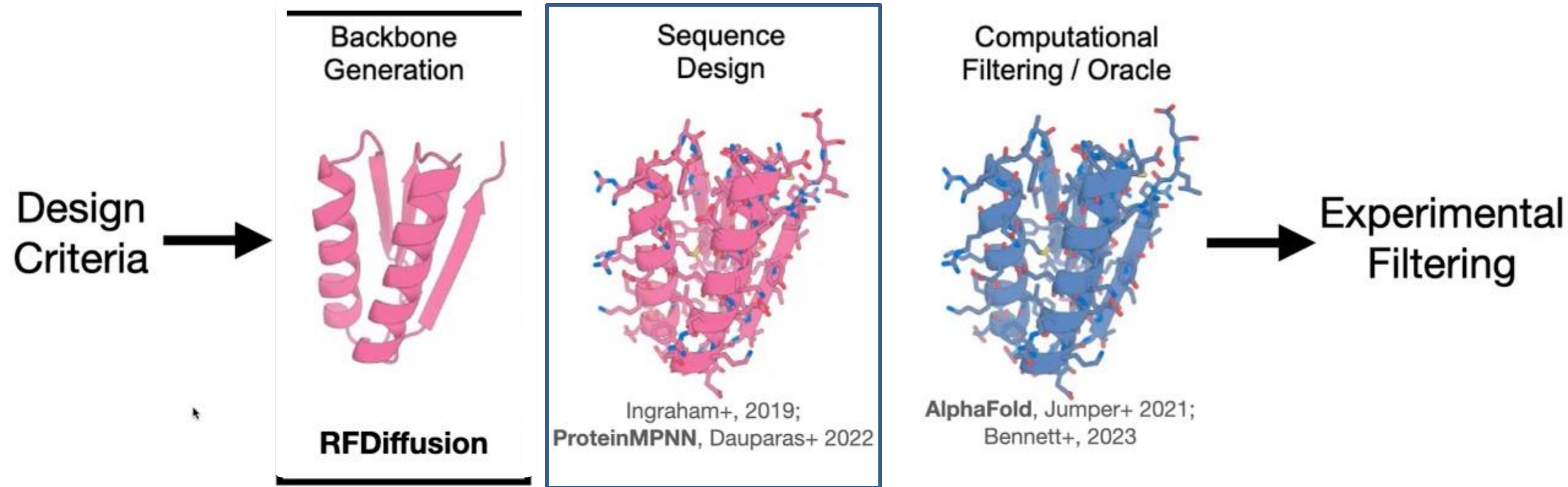
Prize share: 1/4

The Nobel Prize in Chemistry 2024 was divided, one half awarded to David Baker "for computational protein design", the other half jointly to Demis Hassabis and John Jumper "for protein structure prediction"

The Nobel Prize in Chemistry 2024

- The Nobel Prize in Chemistry 2024 is about **proteins, life's ingenious chemical tools**. David Baker has succeeded with the **almost impossible feat of building entirely new kinds of proteins**. Demis Hassabis and John Jumper have developed an AI model to solve a 50-year-old problem: **predicting proteins' complex structures**. These discoveries hold enormous potential. ...
- Life could not exist without proteins. That we can now predict protein structures and design our own proteins confers the greatest benefit to humankind.

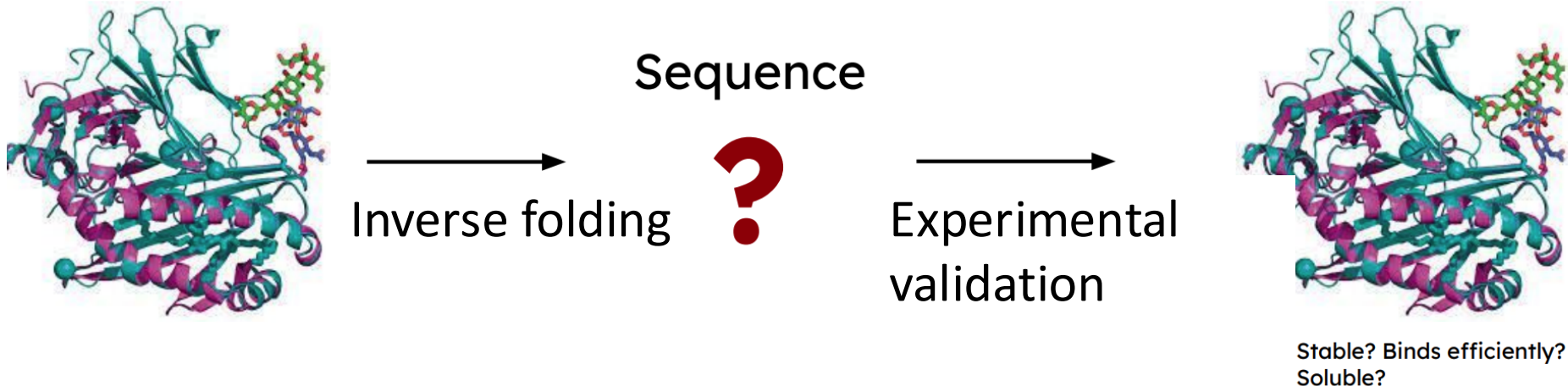
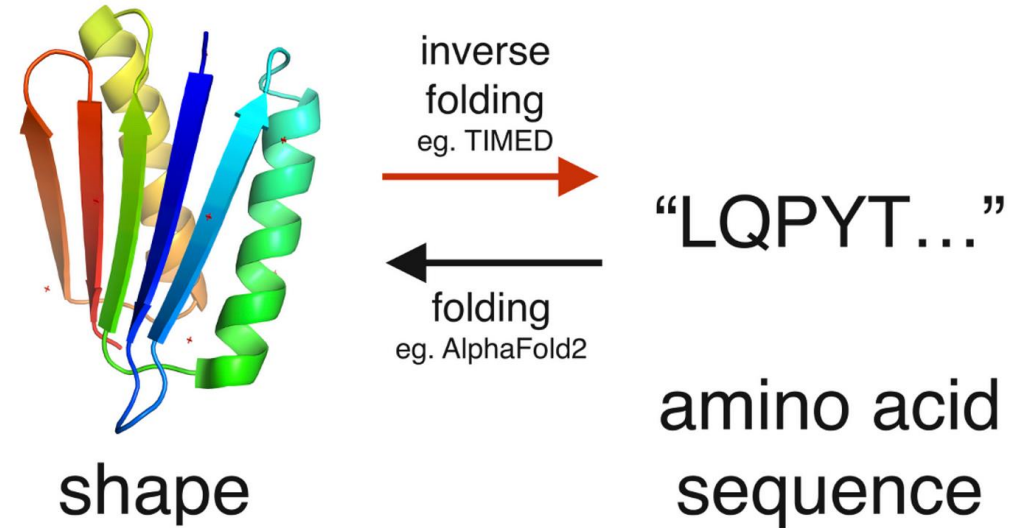
De Novo Protein Design Workflow



- Backbones must be (1) physically **realizable**, (2) **functional**, and (3) **diverse**

Protein Folding & Inverse Folding

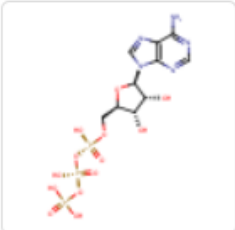
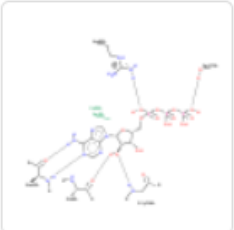
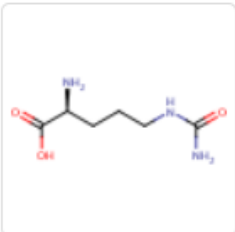
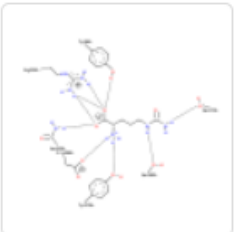
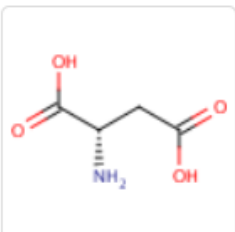
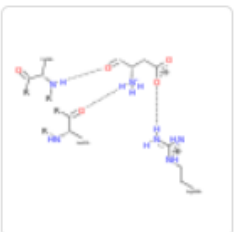
- **Folding:** given amino acid sequence, predict protein structure
- **Inverse folding:** given desired protein structure, find amino acid sequence



The Key Dataset: PDB

Small Molecules

Ligands **3 Unique**

ID	Chains	Name / Formula / InChI Key	2D Diagram & Interactions		3D Interactions
ATP Query on ATP Download SDF File  Download CCD File 	A, B, C, D	ADENOSINE-5'-TRIPHOSPHATE <chem>C10H16N5O13P3</chem> ZKHQWZAMYRWXGA-KQYNXXCUSA-N			Ligand Explorer NGL Binding Pocket (JSmol) Electron Density (JSmol)
CIR Query on CIR Download SDF File  Download CCD File 	A, B, C, D	CITRULLINE <chem>C6H13N3O3</chem> RHGKLRLOHDJJDR-BYPYZUCNSA-N			Ligand Explorer NGL Binding Pocket (JSmol) Electron Density (JSmol)
ASP Query on ASP Download SDF File  Download CCD File 	A	ASPARTIC ACID <chem>C4H7NO4</chem> CKLJMWZIZZHCS-REOHCLBHSA-N			Ligand Explorer NGL Binding Pocket (JSmol) Electron Density (JSmol)

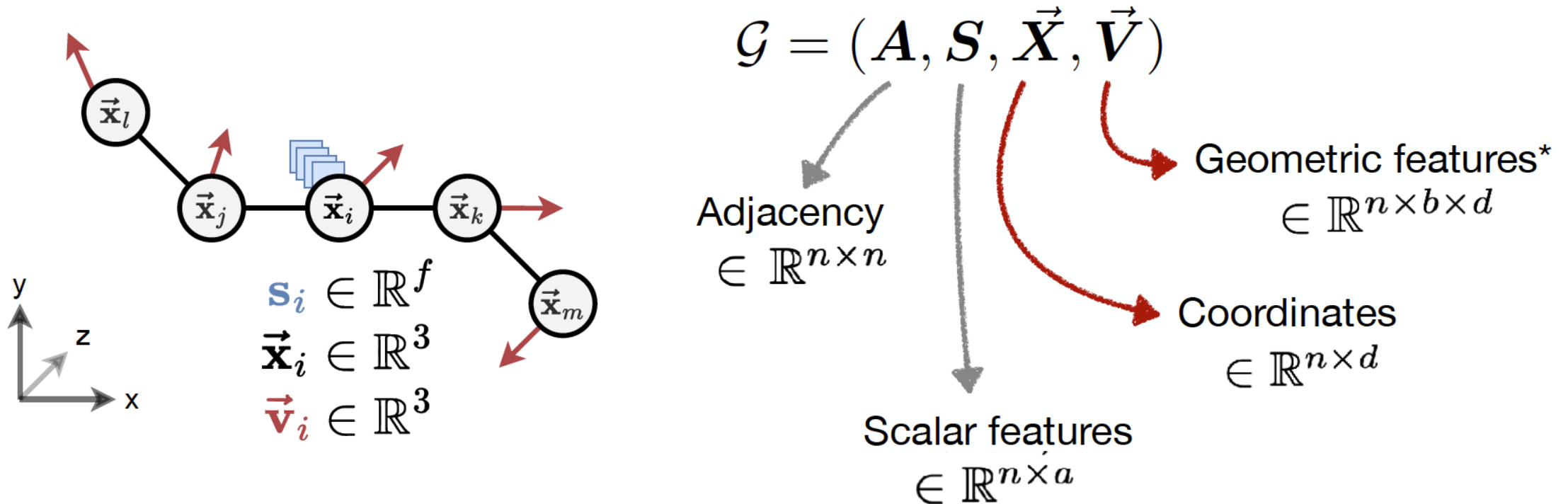
PDB Example

	id	type								x	y	z								
ATOM	1	N	N	.	VAL	A	1	1	?	6.204	16.869	4.854	1.00	49.05	?	1	VAL	A	N	1
ATOM	2	C	CA	.	VAL	A	1	1	?	6.913	17.759	4.607	1.00	43.14	?	1	VAL	A	CA	1
ATOM	3	C	C	.	VAL	A	1	1	?	8.504	17.378	4.797	1.00	24.80	?	1	VAL	A	C	1
ATOM	4	O	O	.	VAL	A	1	1	?	8.805	17.011	5.943	1.00	37.68	?	1	VAL	A	O	1
ATOM	5	C	CB	.	VAL	A	1	1	?	6.369	19.044	5.810	1.00	72.12	?	1	VAL	A	CB	1
ATOM	6	C	CG1	.	VAL	A	1	1	?	7.009	20.127	5.418	1.00	61.79	?	1	VAL	A	CG1	1
ATOM	7	C	CG2	.	VAL	A	1	1	?	5.246	18.533	5.681	1.00	80.12	?	1	VAL	A	CG2	1

Proteins as Geometric graphs

Each node is:

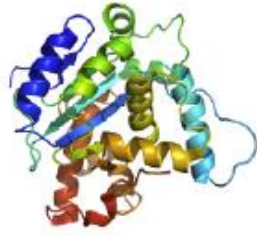
- **embedded in Euclidean space** e.g. atoms in 3D
- **decorated with geometric attributes** s.a. velocity



Real-world Geometric Graphs



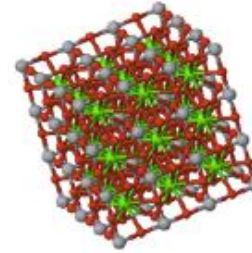
Small
Molecules



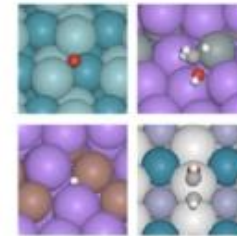
Proteins



DNA/RNA



Inorganic
Crystals



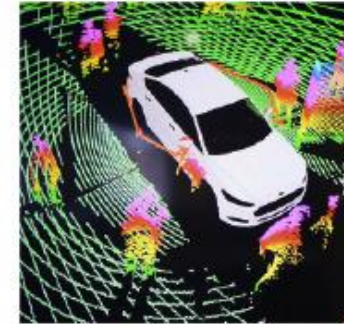
Catalysis
Systems



Transportation &
Logistics



Robotic
Navigation



3D Computer
Vision

GNN Applications: Graph for Protein Design

Inverse Protein Folding: ProteinMPNN

ProteinMPNN

RESEARCH

PROTEIN DESIGN

Robust deep learning-based protein sequence design using ProteinMPNN

J. Dauparas^{1,2}, I. Anishchenko^{1,2}, N. Bennett^{1,2,3}, H. Bai^{1,2,4}, R. J. Ragotte^{1,2}, L. F. Milles^{1,2}, B. I. M. Wicky^{1,2}, A. Courbet^{1,2,4}, R. J. de Haas⁵, N. Bethel^{1,2,4}, P. J. Y. Leung^{1,2,3}, T. F. Huddy^{1,2}, S. Pellock^{1,2}, D. Tischer^{1,2}, F. Chan^{1,2}, B. Koepnick^{1,2}, H. Nguyen^{1,2}, A. Kang^{1,2}, B. Sankaran⁶, A. K. Bera^{1,2}, N. P. King^{1,2}, D. Baker^{1,2,4*}

- ProteinMPNN is to protein design what AlphaFold was to protein structure prediction – David Baker

ProteinMPNN's Empirical Verification

RESEARCH

PROTEIN DESIGN

Hallucinating symmetric protein assemblies

B. I. M. Wicky^{1,2†}, L. F. Milles^{1,2†}, A. Courbet^{1,2,3†}, R. J. Ragotte^{1,2}, J. Dauparas^{1,2}, E. Kinfu^{1,2}, S. Tipps^{1,2}, R. D. Kibler^{1,2}, M. Baek^{1,2}, F. DiMaio^{1,2}, X. Li^{1,2}, L. Carter^{1,2}, A. Kang^{1,2}, H. Nguyen^{1,2}, A. K. Bera^{1,2}, D. Baker^{1,2,3*}

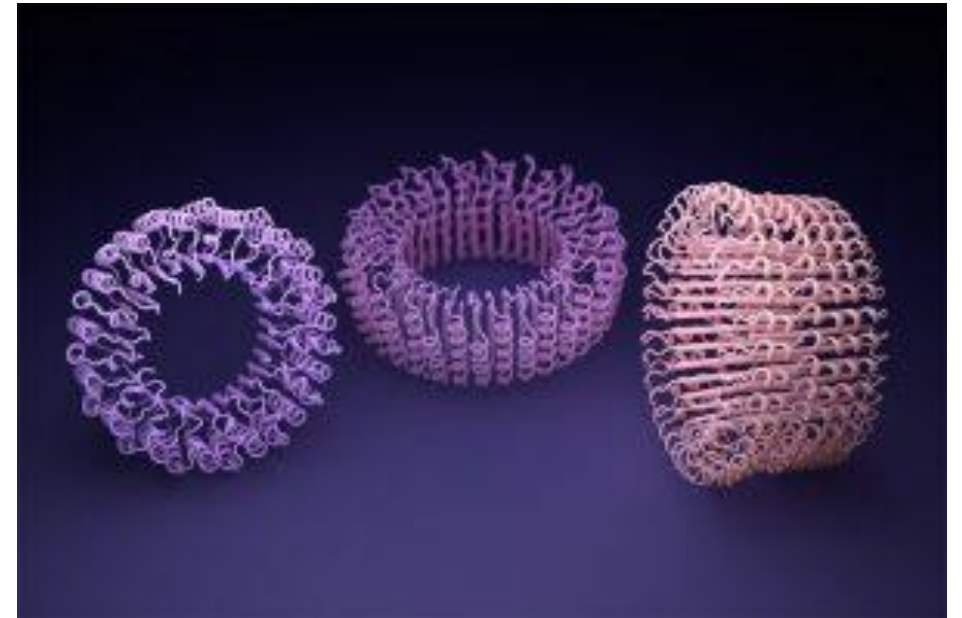
- ProteinMPNN together with the other new machine learning tools could reliably generate proteins that functioned in the laboratory

ProteinMPNN's Potential

NEWS | 15 September 2022 | Correction [21 September 2022](#)

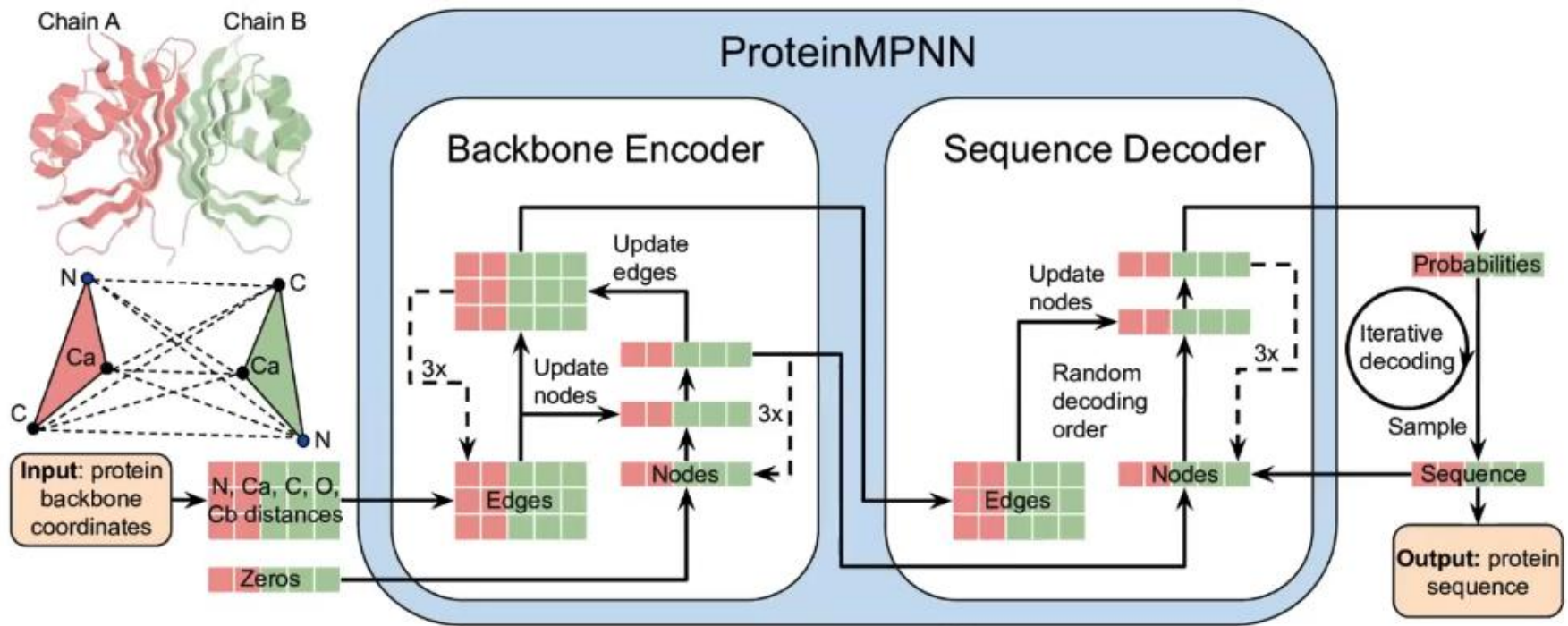
Scientists are using AI to dream up revolutionary new proteins

Huge advances in artificial intelligence mean researchers can design completely original molecules in seconds instead of months.



ProteinMPNN Framework

Structure



Sequence

Fixed left to right decoding

1	2	3	4	5
		G	E	

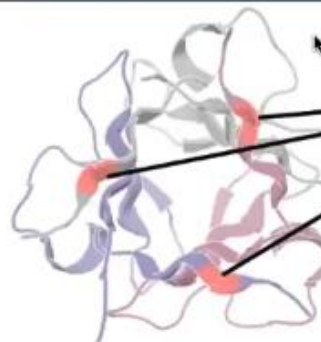
Chain A

ProteinMPNN decoding

3	5	1	2	4
		G	E	

Chain A

- autoregressive decoding order
- fixed amino acids (context)
- sequence context not used
- sequence context used



Tied across chains

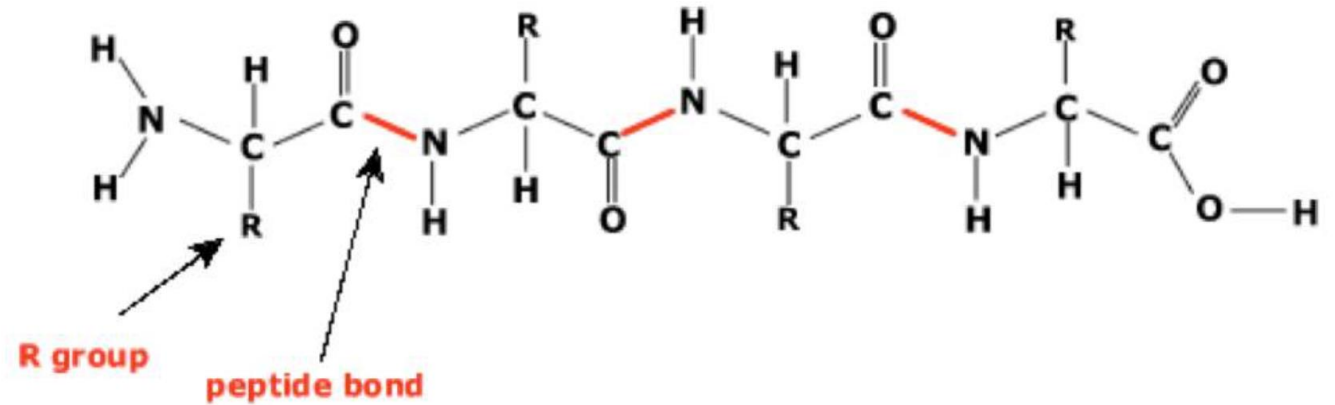
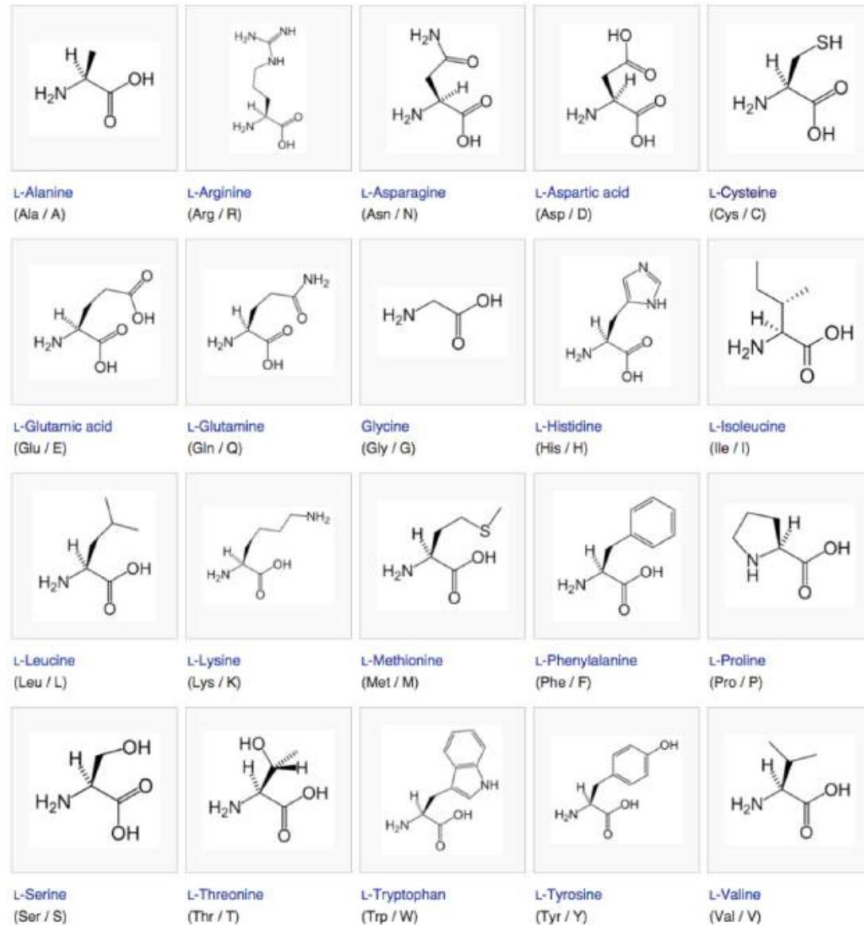
1	3	2		1	3	2		1	3	2	
E	P			E	P			E	P		

Chain A

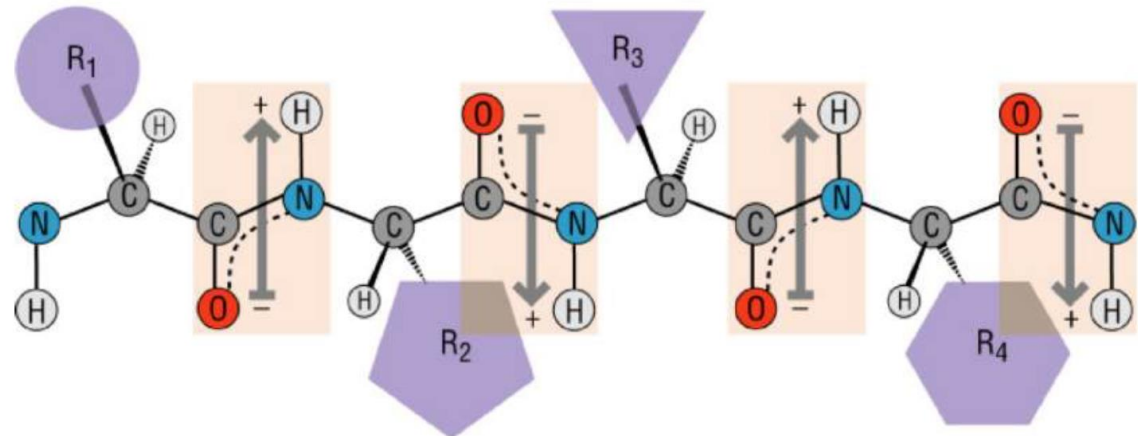
Chain B

Chain C

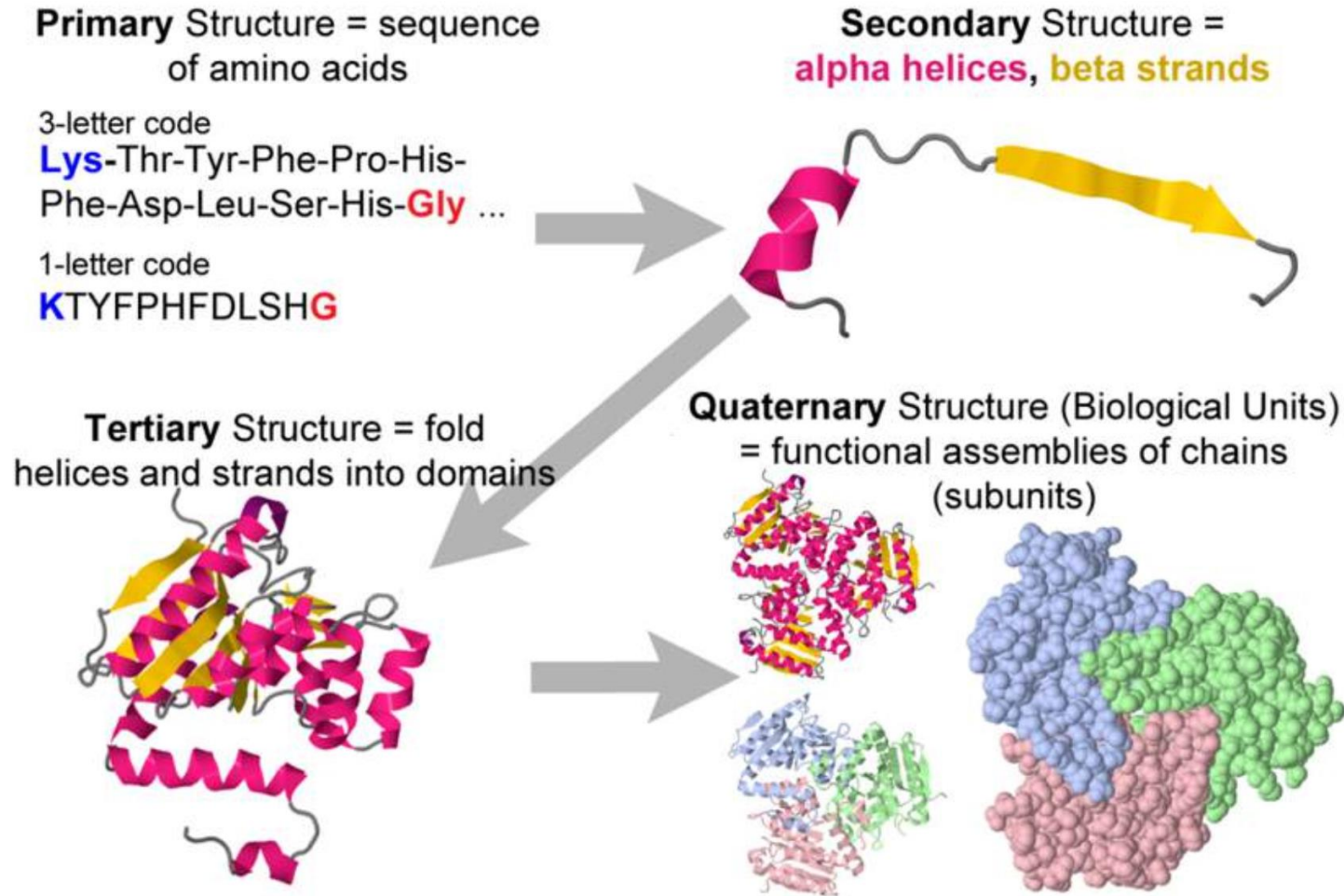
Background: Protein Backbones



From [Protein Structure and Function](#) by Gregory A Petsko and Dagmar Ringe

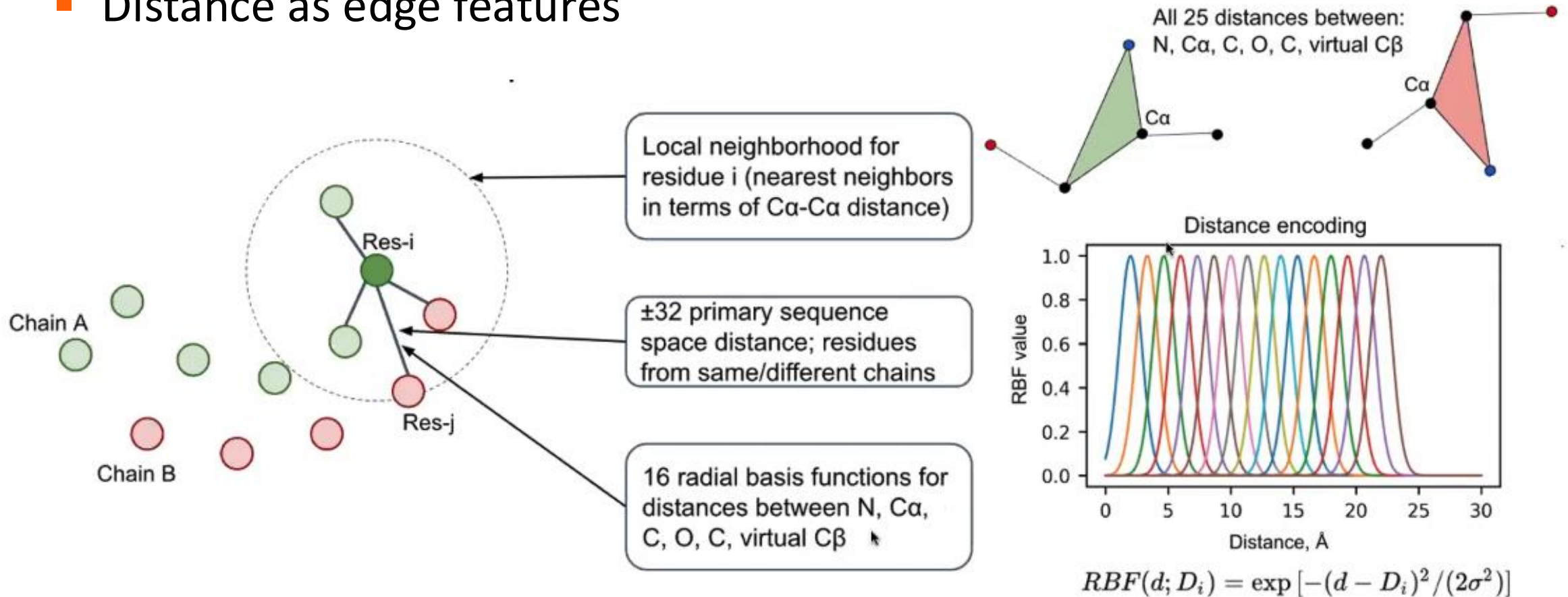


Background: Levels of Protein Structure

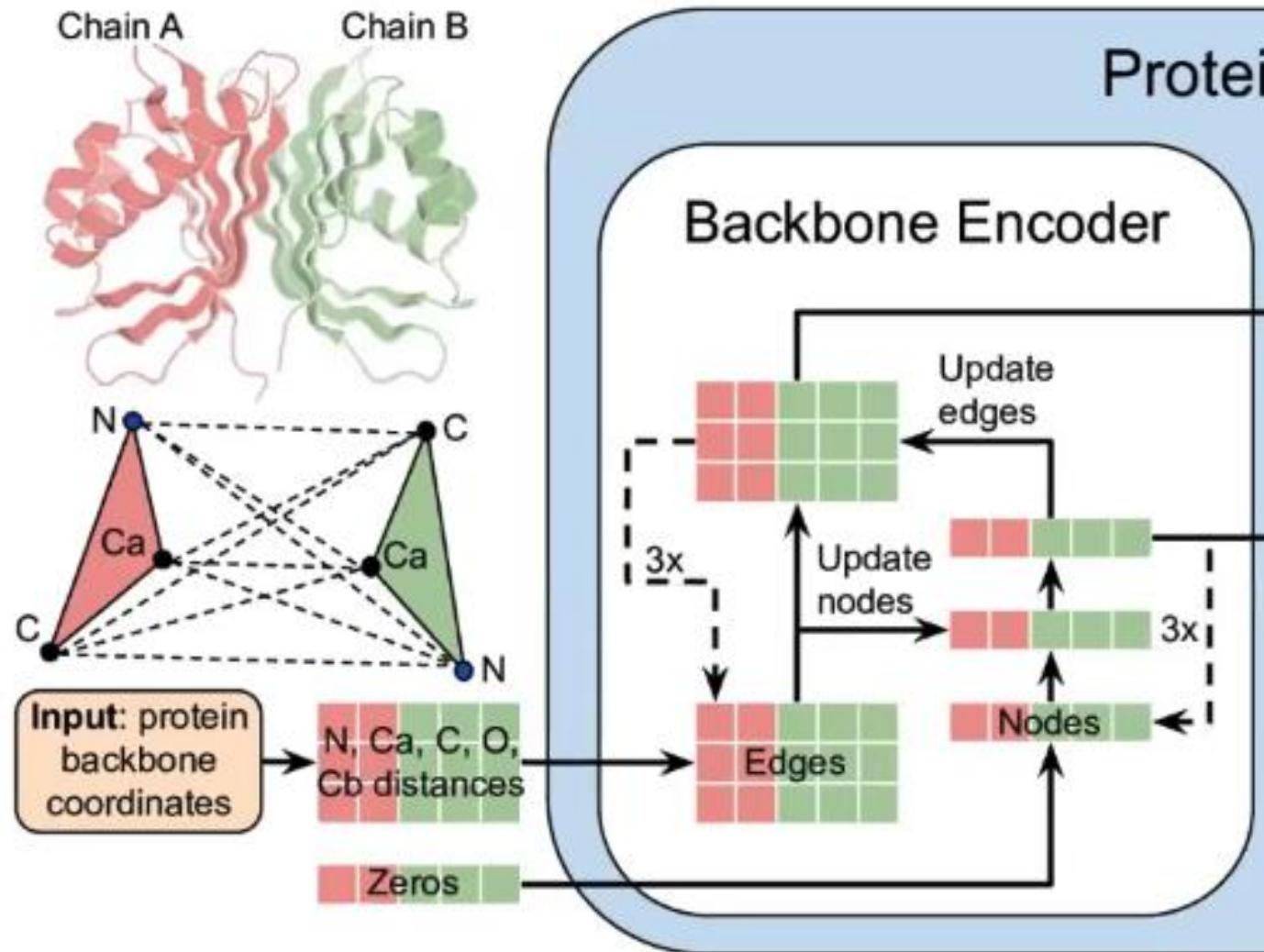


Input Featurization in ProteinMPNN

- Distance as edge features



Structure Encoder in ProteinMPNN



Pseudocode of ProteinMPNN Encoder

- 3-layer encoder -> 3-hop information

Pseudocode for the encoder layer (V - node features, E - edge features):

def encoder_layer_forward(V, E):

$M_{ij} = \text{MLP}[V_i, V_j, E_{ij}]$  Get intermediate representation or “message” based on information of neighbors and edges for node i

$dV_i = \text{Sum}_j [M_{ij}]$  Sum messages across all neighbors

$V_i = \text{LayerNorm}[V_i + \text{Dropout}(dV_i)]$ 

$dV_i = \text{FeedForward}[V_i]$ Updates node representations

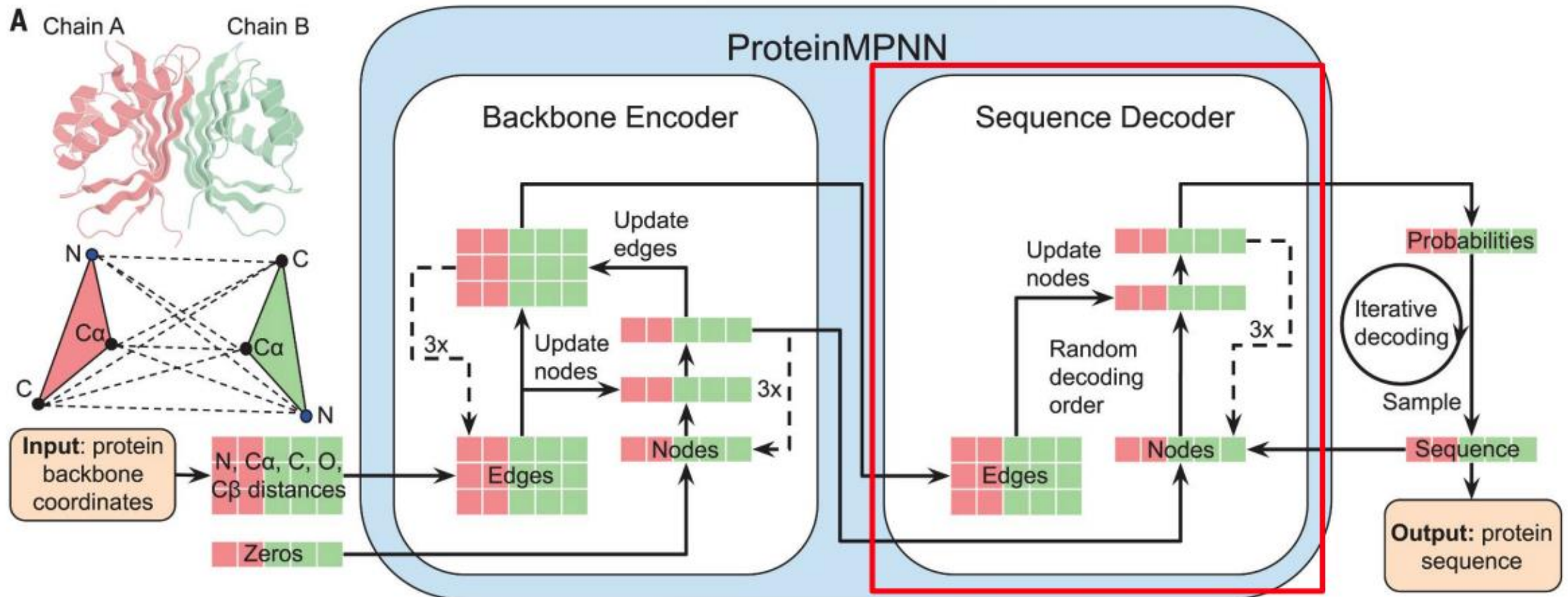
$V_i = \text{LayerNorm}[V_i + \text{Dropout}(dV_i)]$

$dE_{ij} = \text{MLP}[V_i, V_j, E_{ij}]$  Updates edges representations based on new node representations

$E_{ij} = \text{LayerNorm}[E_{ij} + \text{Dropout}(dE_{ij})]$

return V, E

Autoregressive Decoder in ProteinMPNN



Pseudocode of ProteinMPNN Decoder

- 3-layer decoder -> additional 3-hop information

Pseudocode for the decoder layer (V - node features, E - edge features, S - sequence features, $mask$ - autoregressive mask):

Use information about previous time step to predict at current step

def decoder_layer_forward(V , E , S , $mask$):

$E_{ij} = \text{Concat}[E_{ij}, S_j] * mask_{ij} + \text{Concat}[E_{ij}, 0.0 * S_j] * (1 - mask_{ij})$  Add edge to sequence features together

$M_{ij} = \text{MLP}[V_i, V_j, E_{ij}]$

$dV_i = \text{Sum}_j [M_{ij}]$

$V_i = \text{LayerNorm}[V_i + \text{Dropout}(dV_i)]$

$dV_i = \text{FeedForward}[V_i]$

$V_i = \text{LayerNorm}[V_i + \text{Dropout}(dV_i)]$

return V

Use encoded neighbor embeddings to update current embeddings

ProteinMPNN Uses Random Decoding Order

- Due to graph permutation equivariance
 - Recall in GraphRNN, BFS order could be better than random order

Fixed left to right decoding



Chain A

ProteinMPNN decoding

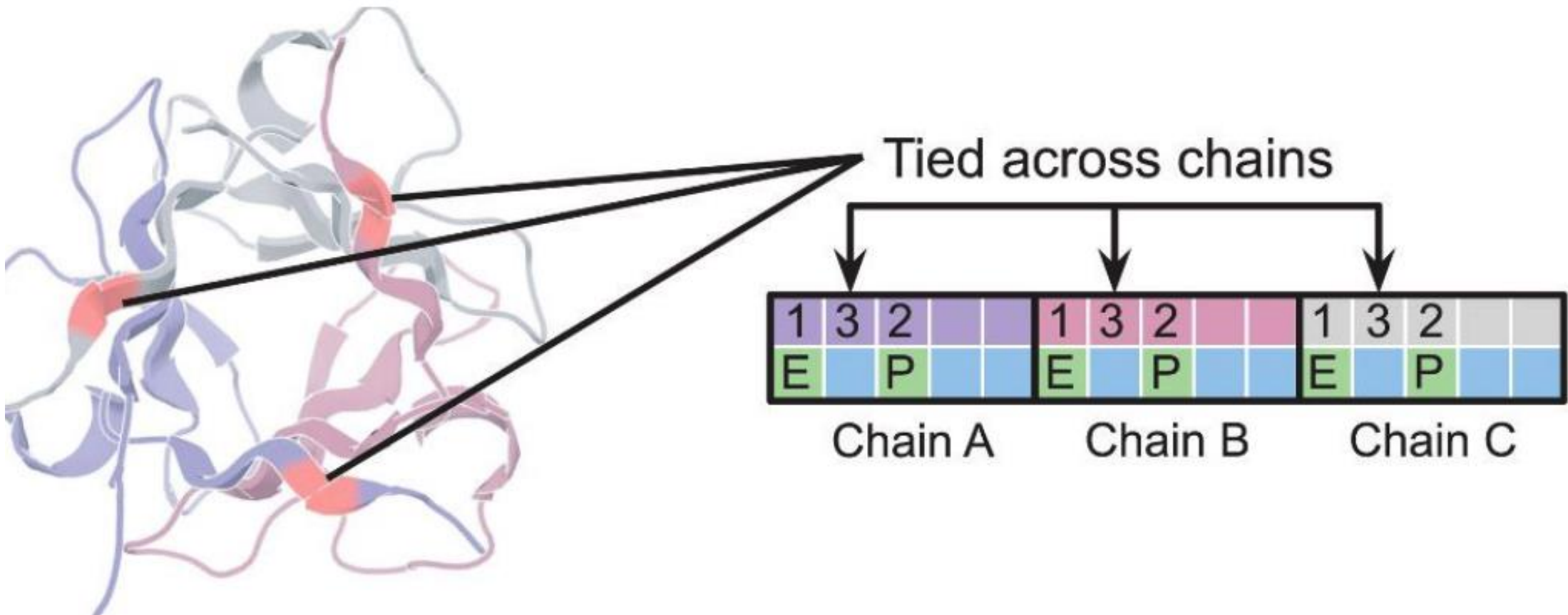


Chain A

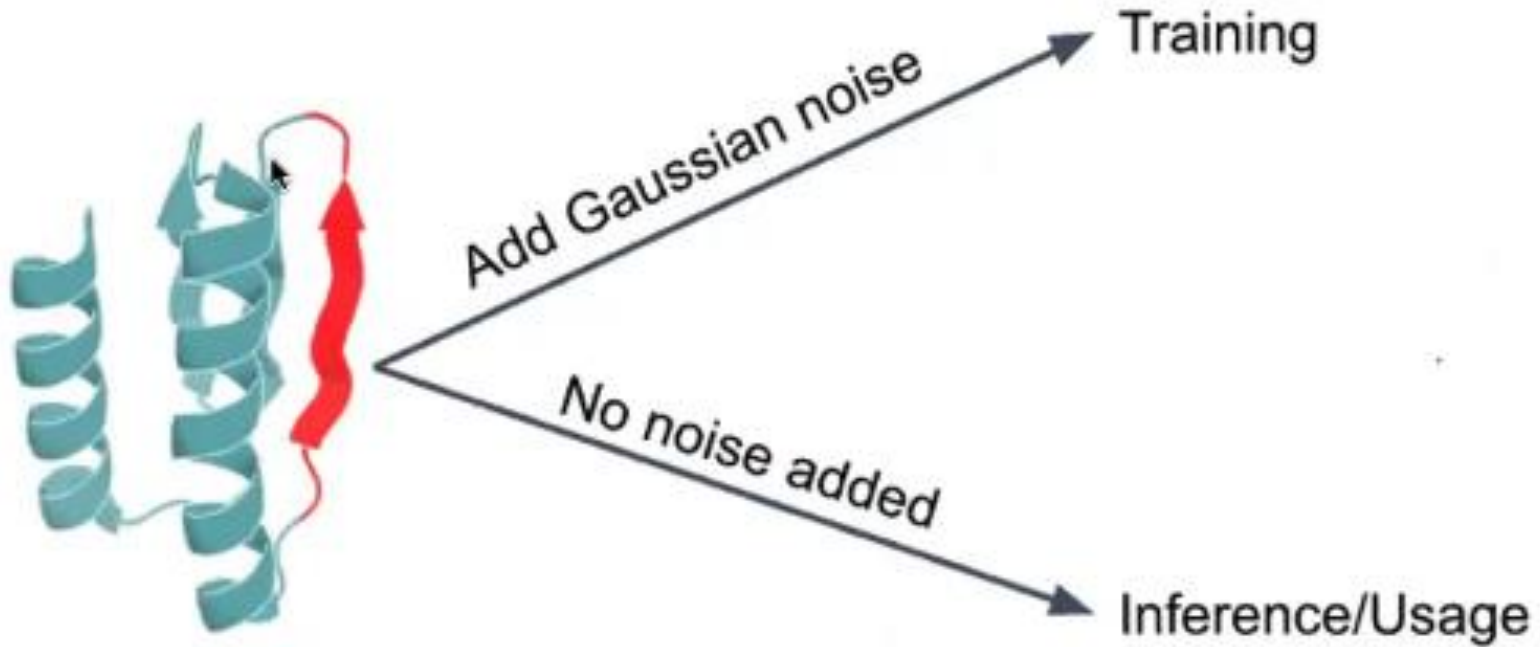
- autoregressive decoding order
- fixed amino acids (context)
- sequence context not used
- sequence context used

ProteinMPNN Uses Positional Coupling for Multichain Predictions

- Ensure certain residues have the same output



Add Noise to Backbone during Training



- Noise added to input edge features
 - Residue distances

How was ProteinMPNN Trained

- Data from PDB (X-ray or cryoEM)
- Random train/val/test split (23358/1464/1529)
 - Different chains from 1 protein must belong to the same split – no leakage
- Training
 - Pick a protein sequence
 - Given the sequence, pick a conformation (coordinates)
 - Loss: classification loss of the residual type
 - Metric: accuracy, runtime

ProteinMPNN Code

- **ProteinMPNN code**
- <https://github.com/dauparas/ProteinMPNN/tree/main>
- **Demo - Let's dive into the code!**
- https://github.com/dauparas/ProteinMPNN/blob/8907e6671bfbfc92303b5f79c4b5e6ce47cdef57/protein_mpnn_utils.py#L1019
- It's fun to learn that they implemented MPNN without PyG

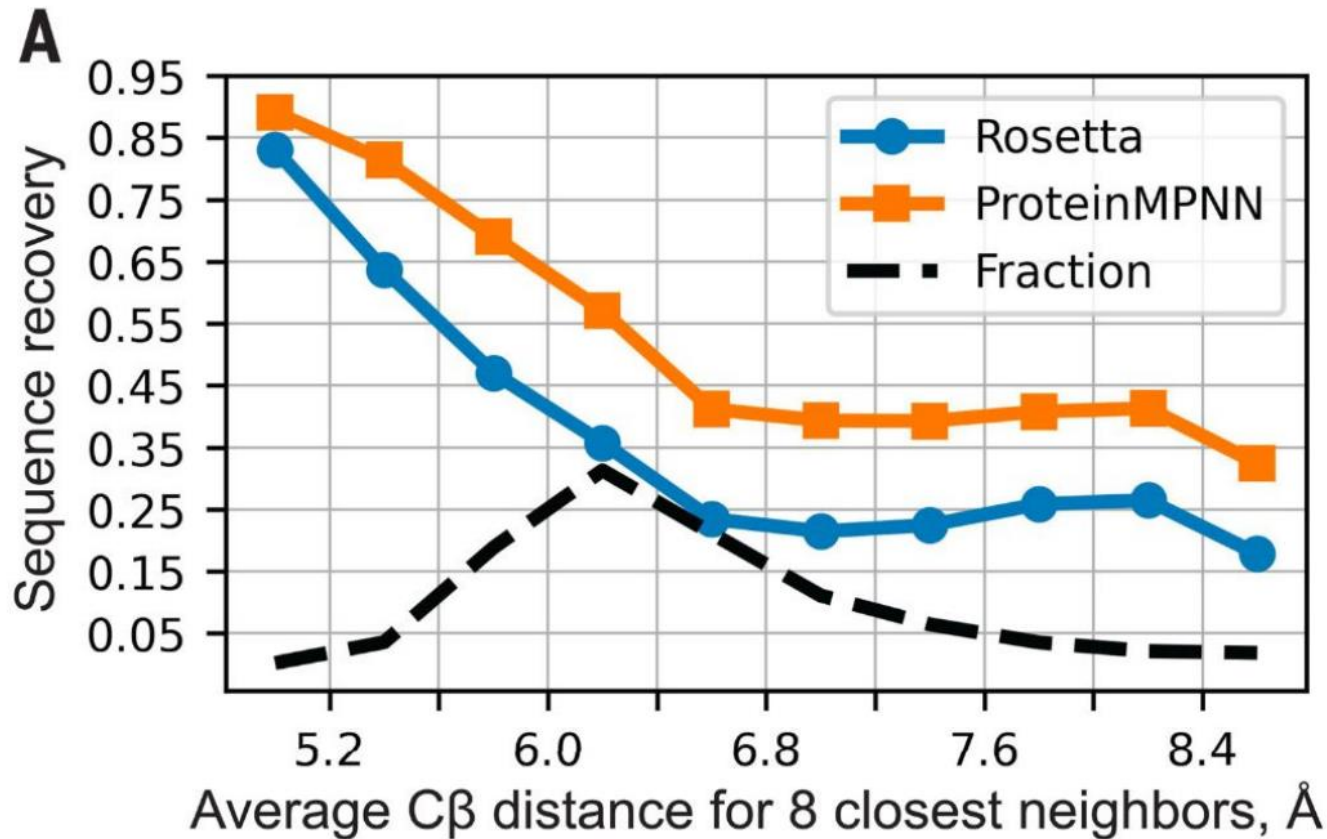
ProteinMPNN Results

- Adding distances as edge features are helpful

Noise level when training: 0.00 Å/0.02 Å	Modification	Number of parameters in millions	PDB test accuracy (%)	PDB test perplexity	AlphaFold model accuracy (%)
Baseline model	None	1.381	41.2/40.1	6.51/6.77	41.4/41.4
Experiment 1	Add N, C α , C, C β , 0 distances	1.430	49.0/46.1	5.03/5.54	45.7/47.4
Experiment 2	Update encoder edges	1.629	43.1/42.0	6.12/6.37	43.3/43.0
Experiment 3	Combine 1 and 2	1.678	50.5/47.3	4.82/5.36	46.3/47.9
Experiment 4	Experiment 3 with random decoding	1.678	50.8/47.9	4.74/5.25	46.9/48.5

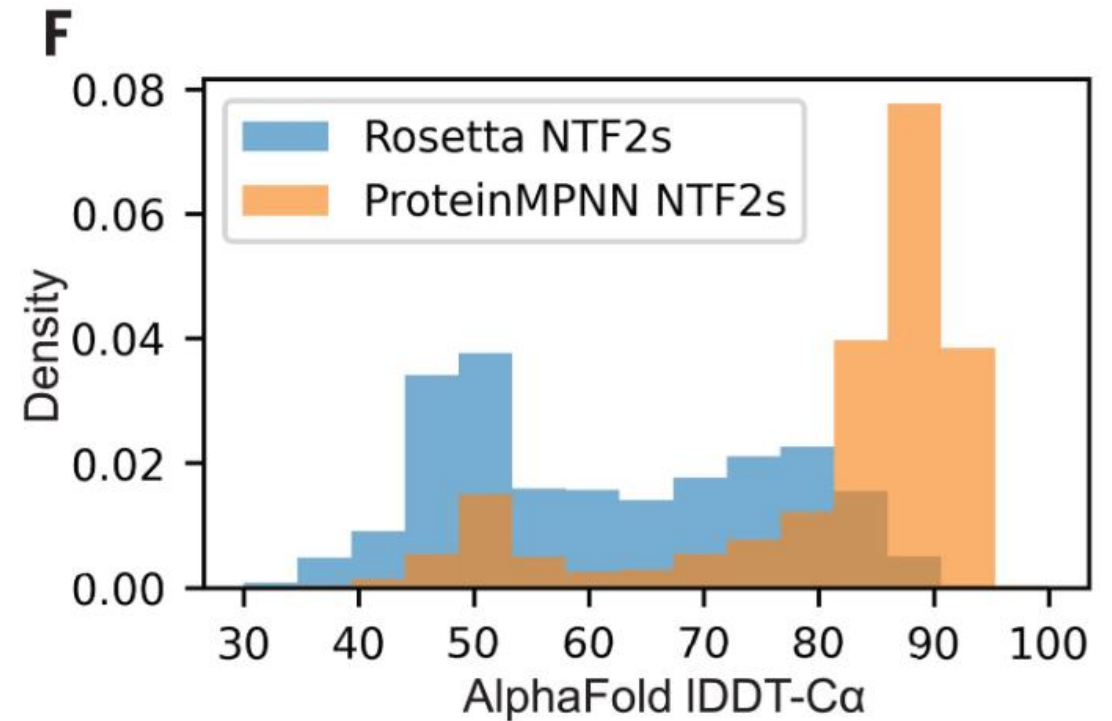
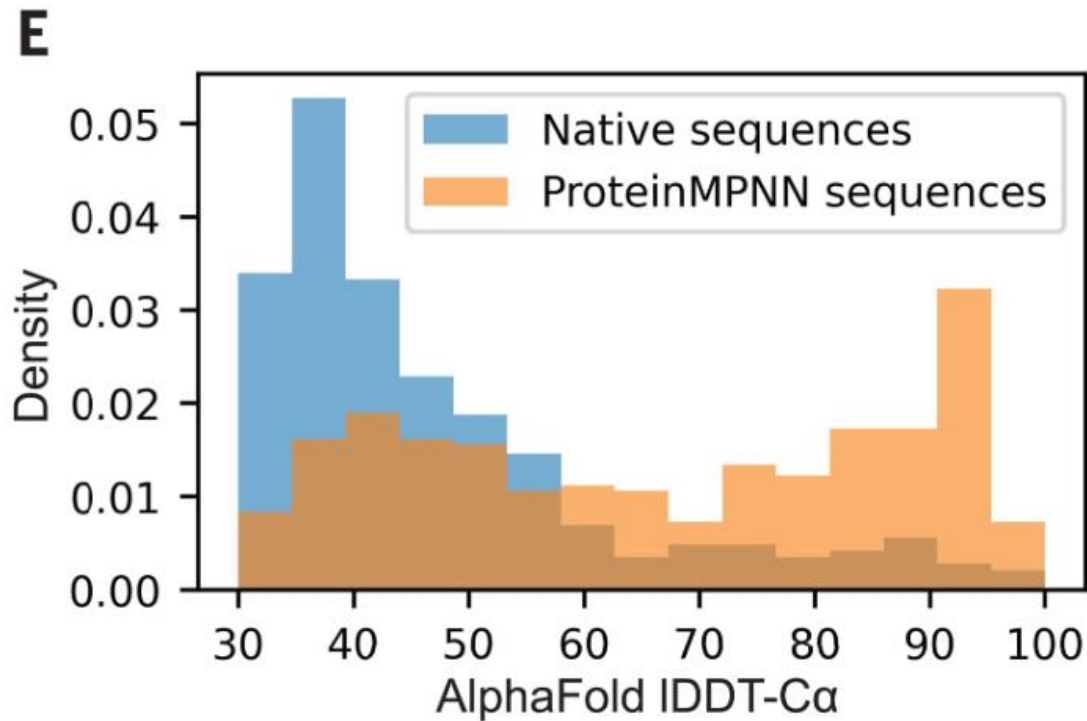
Ingraham et. al

ProteinMPNN is Better than Classic Methods



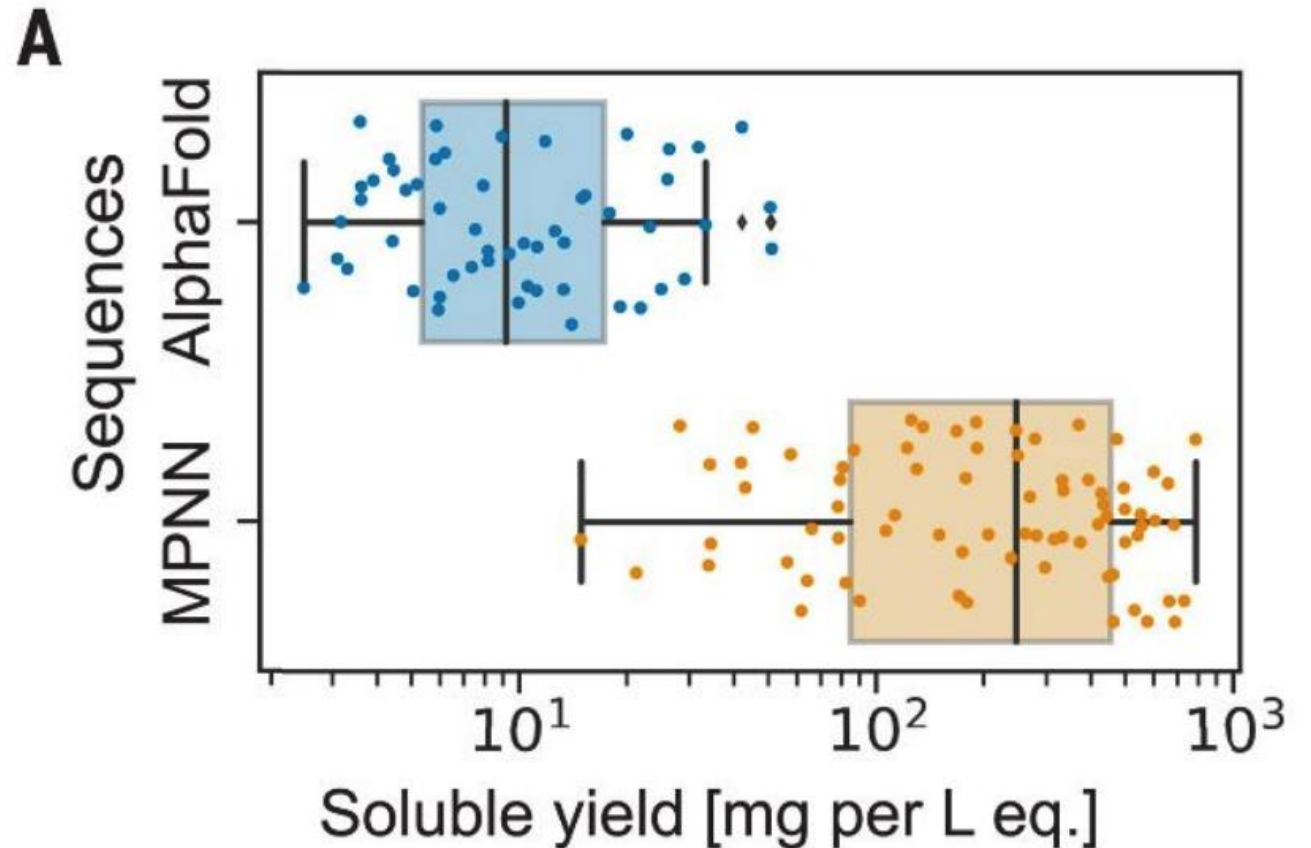
- **Sequence recovery:** 54.29% vs. 32.9%
- **Run time:** 1.2s vs. 258.8s (1 CPU for 100 residues)

In Silico Evaluation of ProteinMPNN

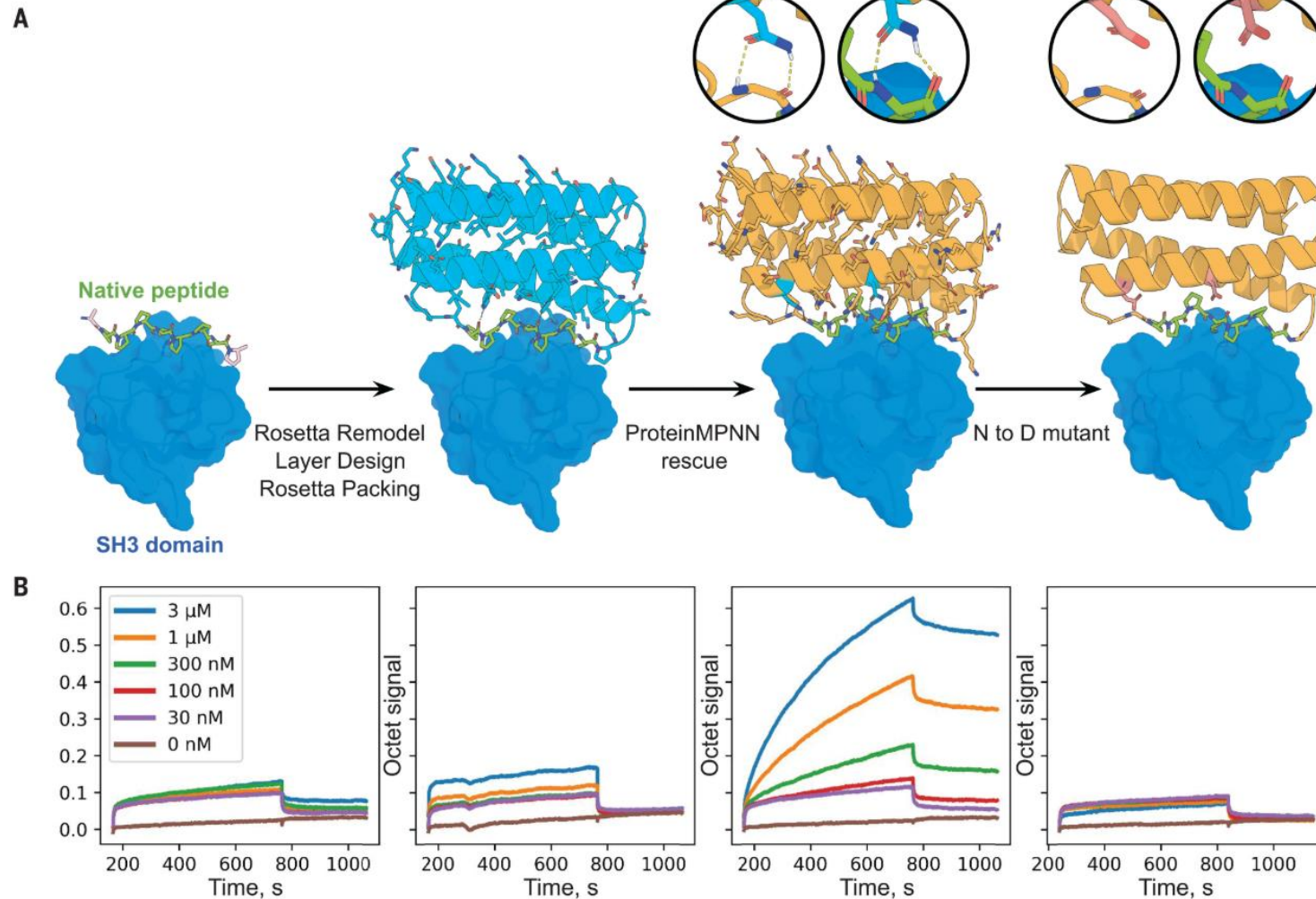


Experimental Validation of ProteinMPNN

1. Network hallucination by AlphaFold to produce backbone set
2. Monte Carlo to generate variety of AlphaFold sequences
3. ProteinMPNN to generate sequences
4. Express these proteins in E. coli



Rescue Failed Design with ProteinMPNN



Summary

- MPNN is effective in protein inverse folding task
 - Node distances as edge features
 - Node & edge embedding update after each layer
 - Encoder decoder design
- ProteinMPNN has yielded significant real-world impact
 - Used in combination with AlphaFold (folding) in protein design
 - Verified in real-world experiments
 - Dr. Baker & AlphaFold won Nobel Prize in Chemistry 2024 together